MALLA REDDY TECHNICAL CAMPUS
A Constituent Unit of MALLA REDDY VISHWAVIDYAPEETH
Deemed to be University
Approved by UGC & AICTE New Delhi
Maisammaguda, Medchal (Dist), Hyderabad - 500100, Telangana

Department of Data Science

PROBLEM – TO - PRODUCT HACKATHON (P2P - BEH 2026)

PROJECT DOCUMENTATION

Date: 10-10-2026 | Venue: 1st Floor, A & B Blocks

Field	Details
Project Title	SkillCircle: A Neighborhood Skill Exchange Platform
Team Name / Team No.	SkillCircle / Team No. 17
Year / Semester / Section	II Year B.Tech / Semester and section not provided
Algorithm Paradigm Used	Graph (conceptual neighborhood skill network); linear search/filtering in prototype
Programming Language / Tools	HTML, CSS, JavaScript; code editor and web browser (specific editor not specified)
Faculty Mentor	Noureen Tabassum

Team Members

S.No	Name of the Student	Roll No.	Role
1	Roopika Ishwarya Gangireddy	25MVCSDR0159	Team Lead; overall project coordination and presentation
2	Rupasri Racharla	25MVCSDR0158	UI design and frontend layout
4	Suguna Rudaram	25MVCSDR0160	Skill listing and category organization
5	S. Loith	25MVCSDR0161	Testing and documentation and Prototype testing

1. Problem Statement

State the problem clearly, with its inputs, outputs and constraints.

SkillCircle addresses the difficulty of discovering and accessing useful skills within a neighborhood. Residents may rely on personal contacts, scattered posts, or formal classes, which can make local knowledge hard to find and lessons costly or inconvenient. Inputs include a search term or category, a skill-offering form (title, category, description), and a learning-request action. Outputs are matching skill listings, newly added demo listings, and a displayed request status. The current prototype uses sample data and browser-side interactions; data is not shared across users or permanently stored.

2. Objectives

List 2–4 specific objectives of the project.

1. Let residents browse skill listings and discover skills available nearby.
2. Provide search and category filtering to narrow relevant offerings.
3. Allow users to add a skill using a title, category, and description.
4. Demonstrate a learning-request flow and show its status.

3. Proposed Approach

Explain the algorithm design paradigm chosen and why it suits this problem. Mention alternatives considered.

The project models the neighborhood as a network of people and skills, so a graph is a useful conceptual representation of the local knowledge network. In the current frontend demo, however, the implemented core operations are simpler: JavaScript searches and filters a sample list, validates form input, adds a listing to in-memory data, and updates a demo request status. Linear scanning is suitable for the small prototype dataset because it is straightforward and easy to demonstrate. Alternatives include a graph traversal for richer person-to-skill relationships or indexed search for larger datasets; these are not implemented in the current prototype.

4. Algorithm / Pseudocode

Write the step-by-step algorithm or pseudocode. Add a flowchart if needed.

ALGORITHM: SEARCH AND FILTER SKILLS
Input: sample skill list L, optional search term q, optional category c
1. Start with an empty result list R.
2. For each skill s in L:
   a. Check whether s matches the selected category, if one is selected.
   b. Check whether s matches q in the searchable text, if q is provided.
   c. If both applicable checks pass, append s to R.
3. Display R in the browser interface.
4. If the user submits a valid skill form, create a skill record and add it to the in-memory list.
5. If the user requests to learn a skill, record the demo request interaction and display its status.
Output: matching listings and updated demo interface.

Note: This pseudocode summarizes the workflow described in the presentation; exact function names and implementation details were not supplied.

5. Complexity Analysis

Give the time and space complexity, with justification.

Complexity	Best Case	Average Case	Worst Case
Time	O(1) if no records are scanned	O(n)	O(n)
Space	O(1)	O(n) for result list	O(n) for result list

6. Implementation

Describe the main modules or functions, the data structures used and the tools or platform.

Modules and tools:
• Browse and display: renders sample skill listings.
• Search and filter: JavaScript narrows listings by search term or category.
• Offer a skill: validates title, category, and description, then adds a listing to in-memory demo data.
• Request learning: submits a demo request and displays its status.
• Interface: HTML provides page structure; CSS handles layout, styling, and responsive presentation; JavaScript handles browser-side interactions.

Data structures: a list/array of sample skill records and temporary in-memory state for listings and demo requests. The presentation does not specify the exact data structure or function names. React, Node.js/Express, and Supabase/PostgreSQL are possible future technologies, not part of the current prototype.

7. Results and Test Cases

Give sample inputs and outputs and attach screenshots. Compare performance with a naive approach, if applicable.

Illustrative test cases based on the documented prototype workflow (actual execution results/screenshots were not included in the presentation):

Test 1: Search term = “cook” | Expected: listings matching the term are displayed.
Test 2: Category = “Music” | Expected: only listings in the Music category are displayed.
Test 3: Submit a skill with title, category, and description | Expected: a new listing appears in the current browser demo.
Test 4: Submit a learning request for a listing | Expected: the interface displays a demo request status.
Test 5: Submit incomplete skill details | Expected: form validation prevents or flags an incomplete submission.

Performance: searching/filtering scans the sample list once (O(n)); this is simple and adequate for a small demo. No measured comparison with a naive or optimized implementation is provided. The current data is in memory and does not persist or synchronize across users.

8. Conclusion and Future Scope

Summarise the outcome and the improvements possible.

SkillCircle demonstrates a lightweight neighborhood skill-exchange interface where users can browse listings, search or filter by category, offer a skill, and submit a learning request. The project addresses the discovery gap around local knowledge, but the current version is a frontend demonstration with sample data and browser-side interactions only.

Future scope: add user accounts and authentication, persistent shared storage, a backend API, real messaging and notifications, stronger validation and moderation, and improved search. The presentation identifies React, Node.js/Express, and Supabase/PostgreSQL as possible future technologies.

9. References

List textbooks, research papers and online resources (IEEE format).

[1] Mozilla Developer Network, “HTML: HyperText Markup Language,” MDN Web Docs. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/HTML. Accessed: Oct. 10, 2026.
[2] Mozilla Developer Network, “CSS: Cascading Style Sheets,” MDN Web Docs. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/CSS. Accessed: Oct. 10, 2026.
[3] Mozilla Developer Network, “JavaScript,” MDN Web Docs. [Online]. Available:

https://developer.mozilla.org/en-US/docs/Web/JavaScript. Accessed: Oct. 10, 2026.
[4] SkillCircle Hackathon Presentation, “A Neighborhood Skill Exchange Platform,” project presentation, Oct. 10, 2026.